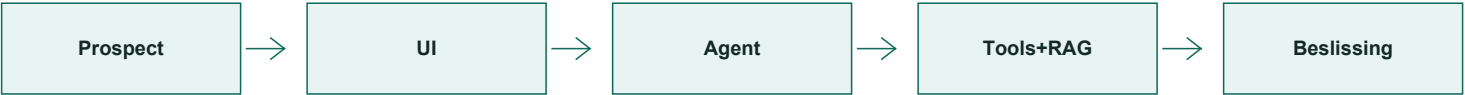


LexIntake

Uitgebreid Leerboek van Begin tot Eind

Alles wat je moet weten over de capstone-opdracht:
Agentic RAG intake screening voor advocatenkantoren
met Agno, LanceDB, OpenAI, Streamlit en CI.

Schema: Grote lijn van het systeem



Schema: Lagenarchitectuur

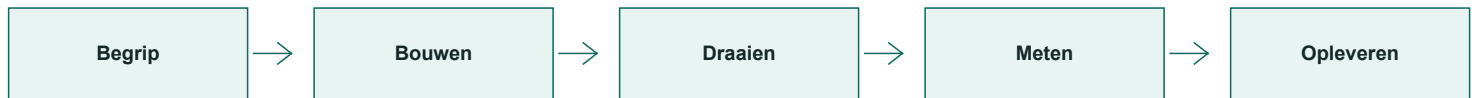
UI	Streamlit interview + quick analysis
Agent	Plan / Retrieve / Tools / Decide / Self-check
Data	LanceDB kb_docs + SQLite entities
KB/ETL	kb/ -> chunk -> embed -> load

*Doel: opdracht, architectuur, repo, start en uitleg leren.
GEEN juridisch advies — alleen educatief materiaal.*

Versie: LexIntake Capstone 2026
Repo: github.com/SteSel123/LexIntake

Inhoudsopgave

Schema: Hoe dit boek is opgebouwd



Schema: Leerpad in 5 delen

Deel A	Probleem, begrippen, eisen
Deel B	KB, ETL, DB, tools, agent
Deel C	Interview, scoring, guardrails
Deel D	Monitoring, eval, UI, CI
Deel E	Voorbeelden, cheatsheets, FAQ

1. Wat is de opdracht en waarom bestaat LexIntake?
2. Belangrijke begrippen (eenvoudig uitgelegd)
3. Capstone-eisen: wat moet je opleveren?
4. Architectuur: het grote plaatje
5. Repository-structuur map voor map
6. Knowledge Base (kb/) bouwen
7. ETL-pipeline stap voor stap
8. Embeddings: hash vs OpenAI
9. Vector database LanceDB
10. Structured database SQLite
11. Agno tools (SOL, conflict, value, route)
12. De Intake Agent (plan → retrieve → tools → decide)
13. Multi-turn interview met prospects
14. Lead scoring & beslissingen
15. Guardrails (juridische veiligheid)
16. Observability & Agno Monitoring
17. Evaluation harness
18. Streamlit UI & demo
19. Configuratie met .env
20. Git-workflow, PR's en CI
21. Stap-voor-stap: systeem starten

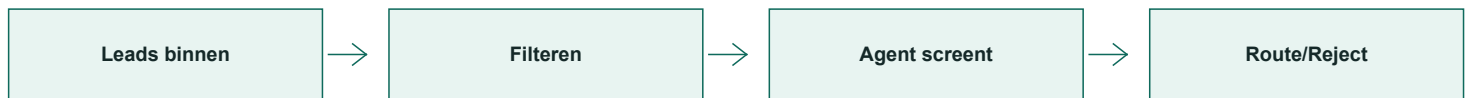
UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

- 22. Demo-video voorbereiden
- 23. Troubleshooting & veelgemaakte fouten
- 24. Oefeningen om te oefenen
- 25. Glossarium & checklist oplevering
- 26. Werkvoorbeelden (PI, SOL, conflict, uncertain)
- 27. Presentatiescript, dataflow, providers, ethiek
- 28. Cheatsheets, zelftest en FAQ-verdieping

Let op: Lees dit boek sequentieel de eerste keer. Daarna kun je per hoofdstuk springen.

Hoofdstuk 1. Wat is de opdracht en waarom bestaat LexIntake?

Schema: Van probleem naar oplossing



Advocatenkantoren krijgen elke dag veel nieuwe leads: via de website, e-mail en telefoonformulieren. Een groot deel van de tijd van een paralegal gaat op aan filteren: verkeerde practice area, verkeerde staat (jurisdiction), verlopen verjaringstermijn (statute of limitations / SOL), belangenconflict, of een zaak die te weinig waard is.

LexIntake is géén chatbot die zomaar praat. Het is een agentic RAG-systeem: een agent die beslist welke kennis hij ophaalt, welke tools hij aanroept, en of hij genoeg bewijs heeft om een lead te scoren en door te sturen.

Wat moet de agent kunnen?

- Prospects interviewen (vragen stellen tot de feiten compleet genoeg zijn)
- Firm knowledge ophalen uit een knowledge base (RAG)
- Tools aanroepen (SOL-check, conflict, case value, routing)
- Leads scoren en een beslissing geven
- Elke beslissing uitleggen met KB-citaten
- Escaleren naar een mens bij onzekerheid

Wat is dit níet?

- Geen juridisch advies aan cliënten
- Geen volledige case-management / CRM-vervanging
- Geen vervanging van een advocaat

Onthoud de disclaimer die overal terugkomt: “This is not legal advice. Consult a licensed attorney.”

Hoofdstuk 2. Belangrijke begrippen (eenvoudig uitgelegd)

Schema: Kernbegrippen als lagen

RAG	Ophalen + genereren met bewijs
Agentic	Plant, kiest tools, beslist
Embedding	Tekst -> vector
Guardrail	Veiligheidsregels

RAG (Retrieval-Augmented Generation)

Eerst relevante tekstfragmenten ophalen uit een kennisbank, daarna pas antwoorden genereren. Zo ‘verzint’ het model minder en kan het citaten geven.

Agentic

Het systeem plant stappen, kiest tools, en beslist of er genoeg info is. Het volgt niet blind één vaste prompt.

Embedding

Een tekst als vector (lijst getallen). Vergelijkbare betekenissen liggen dicht bij elkaar in die ruimte.

Vector database

Database geoptimaliseerd om ‘meest gelijkende’ embeddings te zoeken (semantische search). Bij LexIntake: LanceDB.

Chunk

Klein stukje tekst uit een groter document, handig om te indexeren en te citeren.

Tool calling

De agent roept een functie aan (bijv. `conflict_check`) met gestructureerde input/output.

SOL / Statute of Limitations

Verjaringstermijn: binnen hoeveel tijd moet een claim (globaal) nog geldig zijn.

Guardrail

Veiligheidsregel: disclaimer, geen juridisch advies, escalatie, citaties verplicht.

Deterministisch pad

Zelfde input → zelfde output, zonder externe LLM. Belangrijk voor CI/tests.

Provider-agnostic

Je kunt LLM/embeddings wisselen via config (.env), niet hardcoded aan één vendor.

Hoofdstuk 3. Capstone-eisen: wat moet je opleveren?

Schema: Deliverable-keten



De opdracht-PDF beschrijft leerdoelen, verplichte bouwstenen en deliverables. Hieronder de checklist in mensentaal.

Bouwstenen die er moeten zijn

- Synthetische knowledge base (practice areas, SOL, fees, cases, attorneys, clients, FAQs)
- ETL: extract → clean → dedupe → chunk → metadata → embeddings → load
- ETL-eigenschappen: re-runnable, idempotent, incremental
- Agent-loop: Plan → Retrieve → Tool Call → Decision → Self-check
- Tools: SOL, conflict, estimate_case_value, route_lead (+ optionele fallback)
- Lead output: qualified, lead_score, priority, decision, recommended_attorney
- Guardrails in elk antwoord
- Observability (tokens, cost, latency, tools, retrieval, escalaties, case value)
- Git: main/develop/feature/*, PRs, protected main, GitHub Actions (aanbevolen)
- Stack: Agno + LLM + embeddings + vector DB + SQLite + Streamlit + monitoring
- Evaluation op ~20–30 labeled leads (8 dimensies)

Deliverables

- GitHub repository
- Working system
- Synthetic KB
- Evaluation Report
- Dashboard
- README
- Demo Video (dit moet jij nog opnemen)
- Design Report

Stretch goals (niet verplicht)

- Corrective RAG

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

- Multi-agent team
- Human-in-the-loop console
- Persistent memory
- Uitgebreidere CI/CD (nightly full eval)

LexIntake dekt de MVP-eisen. Stretch is optioneel. De demo-video is de belangrijkste openstaande menselijke deliverable.

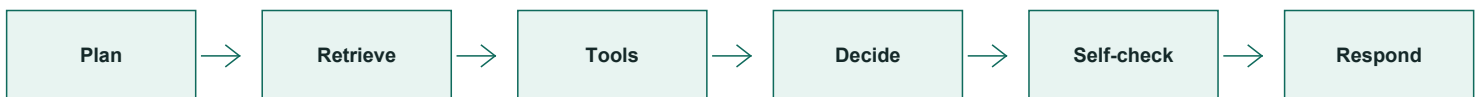
UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 4. Architectuur: het grote plaatje

Schema: Architectuurlagen

UI	Streamlit
Agent	IntakeAgent (Agno)
Tools	SOL / conflict / value / route
Data	LanceDB + SQLite
KB/ETL	kb/ -> embeddings

Schema: Agent-loop



Denk in lagen. Een prospect praat met de UI. De UI praat met de IntakeAgent. De agent haalt kennis uit LanceDB, roept tools aan die SQLite/KB gebruiken, scoort de lead, checkt guardrails, en geeft een uitleg terug.

Prospect / paralegal

```

|
Streamlit UI (interview of quick analysis)
|
IntakeAgent (Agno)
plan -> retrieve -> tools -> decide -> self-check -> respond
|   |   |
LanceDB Agno tools Lead scoring
kb_docs SOL/conflict/value/route
^
ETL (kb/ -> chunks -> embeddings)
SQLite: clients, attorneys, past_cases
  
```

Twee modi

- Live modus: OpenAI embeddings + gpt-4.1 (echte semantiek + uitleg)
- CI/offline modus: hash-embeddings + deterministische agent (geen API-keys)

Dit is bewust zo gebouwd: je kunt lokaal/demo's met keys draaien, terwijl GitHub Actions altijd offline en reproduceerbaar blijft.

Hoofdstuk 5. Repository-structuur map voor map

Schema: Repo-boom

```
LexIntake/
kb/      knowledge base
etl/     extract..embed..load
db/      LanceDB + SQLite
agents/   IntakeAgent + interview
tools/    Agno tools
scoring/  score_lead
monitoring/ metrics + Agno traces
evaluation/ leads + metrics
ui/       Streamlit + demo
docs/     reports + leerboek
.github/  CI smoke-test
```

kb/

Synthetische kennisbank (JSON/MD). Bron van waarheid voor intake-regels.

etl/

Pipeline die KB omzet naar chunks + embeddings en laadt.

db/

LanceDB (vectoren) + SQLite (gestructureerde entities) + load scripts.

tools/

Agno @tool functies: SOL, conflict, value, route, fallback.

agents/

IntakeAgent, interview-sessie, LLM factory, fact parsing.

scoring/

Deterministische score_lead() → beslissing + priority.

monitoring/

JSONL logger, metrics, Streamlit dashboard, Agno traces.

evaluation/

leads.csv, metrics, run_evaluation.py, reports logs.

ui/

Streamlit app + demo scenarios + runner.

docs/

Design report, evaluation report, demo script, dit leerboek.

.github/workflows/

CI smoke-test op pull requests naar main.

Belangrijke losse bestanden

- README.md — setup en overzicht

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

- requirements.txt — Python dependencies
- config.py — leest .env (providers, models, keys)
- .env.example — template zonder geheimen
- .env — jouw lokale secrets (nooit committen)

Hoofdstuk 6. Knowledge Base (kb/) bouwen

Schema: KB-onderdelen

Document	Doel
practice_areas	Scope
acceptance	Qualify rules
sol_tables	Deadlines
past_cases	Valuation
attorneys	Routing
clients	Conflicts
faqs	Grounding

De KB is synthetisch maar realistisch. Ze bestaat zodat de agent iets heeft om op te ‘steunen’ (grounding) in plaats van regels te verzinnen.

Bestanden

- practice_areas.json — vaste lijst van 10 practice areas
- acceptance_criteria.json — wanneer neemt het kantoor een zaak aan?
- fee_structure.json — contingency/hourly/retainer per gebied
- sol_tables.json — SOL-regels per staat/zaaktype
- past_cases.json — vergelijkbare zaken voor valuation
- attorneys.json — specialisatie, ervaring, availability
- clients.json — bestaande cliënten voor conflict checks
- faqs.md — algemene groundingstekst

Practice areas (exact)

Personal Injury, Employment Law, Immigration, Family Law, Criminal Defense, Workers’ Compensation, Medical Malpractice, Product Liability, Civil Rights, Consumer Protection.

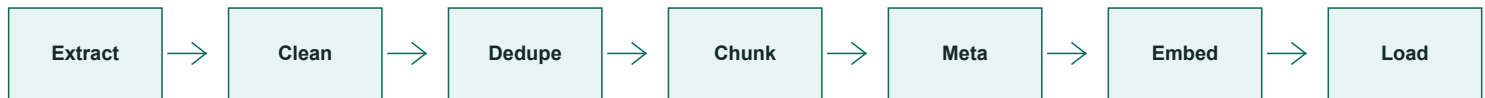
Waarom JSON + Markdown?

JSON is makkelijk te parsen voor tools (SOL/fees/cases). Markdown FAQs zijn goed te chunk’en voor retrieval. De ETL normaliseert alles tot documenten met metadata.

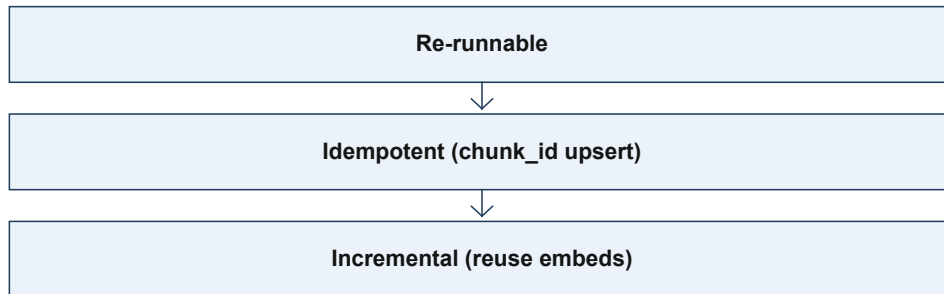
Let op: Wijzig je de KB, dan moet je embeddings opnieuw laden (python db/load_kb_docs.py) zodat LanceDB synchroon blijft.

Hoofdstuk 7. ETL-pipeline stap voor stap

Schema: ETL-pijplijn



Schema: Verplichte ETL-eigenschappen



ETL = Extract, Transform, Load. Bij LexIntake is 'Transform' uitgebreid tot clean, dedupe, chunk, metadata, embeddings.

1. Extract

Lees alle KB-bestanden in als documenten.

2. Clean

Normaliseer whitespace/rommel, maak tekst consistent.

3. Deduplicate

Zelfde inhoud → één document via content_hash.

4. Chunk

Splits lange teksten in stukken met stabiele chunk_id.

5. Metadata

Voeg practice_area, jurisdictions, doc_type, timestamps toe.

6. Embeddings

Maak vectoren (OpenAI of hash).

7. Load

Upsert naar LanceDB kb_docs (en optioneel JSON store).

Verplichte eigenschappen

- Re-runnable: je mag de pipeline steeds opnieuw draaien zonder chaos
- Idempotent: zelfde input convergeert naar dezelfde store-state (upsert op chunk_id)
- Incremental: ongewijzigde chunks kunnen embeddings hergebruiken

Commands

```
python db/init_structured_db.py
python db/load_kb_docs.py
# of via ETL entrypoint:
```

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

```
python etl/load_vector_db.py
```

Hoofdstuk 8. Embeddings: hash vs OpenAI

Schema: Embedding-modi

Pad	Model	Dims	API?
Live	text-emb-3-small	1536	Ja
CI	hash-v1	256	Nee

Schema: Van tekst naar vector



Embeddings zetten tekst om naar getallen. LexIntake heeft twee implementaties achter dezelfde interface `get_embedder()`.

OpenAI text-embedding-3-small (default live)

- Dimensies: 1536
- Echte semantische gelijkenis (“rear-end collision” $\approx$ “car accident injury”)
- Vereist OPENAI_API_KEY
- Gebruikt via Agno OpenAIEmbedder adapter in `etl/embeddings.py`

Hash embedder (CI / offline)

- Dimensies: 256
- Deterministisch uit tokens (geen API)
- Goed genoeg voor smoke tests, minder ‘slim’ semantisch
- Gezet via `LEXINTAKE_EMBEDDING_PROVIDER=hash`

.env voorbeeld

```

LEXINTAKE_EMBEDDING_PROVIDER=openai
LEXINTAKE_EMBEDDING_MODEL=text-embedding-3-small
LEXINTAKE_EMBEDDING_DIMS=1536
OPENAI_API_KEY=sk-...
  
```

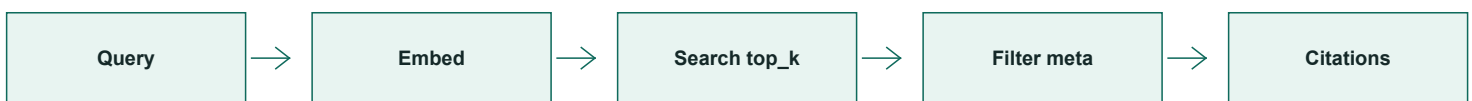
Let op: Wissel je van 256 (hash) naar 1536 (OpenAI), dan herbout LanceDB de tabel vanwege dimensie-mismatch. Dat is normaal.

Hoofdstuk 9. Vector database LanceDB

Schema: kb_docs schema

chunk_id	Primary key voor upsert
text	Chunk inhoud
embedding	Fixed-size float vector
metadata	practice_area / jurisdictions / doc_type

Schema: Retrieval-pad



LanceDB slaat chunks op in collectie kb_docs. Zoeken gebeurt op cosine/vector similarity, met optionele metadata filters.

Schema (kernvelden)

- chunk_id — primaire sleutel voor upsert
- text — chunk tekst
- embedding — fixed-size float vector
- metadata.practice_area
- metadata.jurisdictions (lijst)
- metadata.doc_type

Retrieval in de agent

De agent embedt de query, zoekt top_k (typisch 5–10) en filtert op practice_area / jurisdiction / doc_type. Resultaten worden citaties: chunk_id + metadata.

```

vector = get_embedder().embed([query])[0]
hits = search_kb_docs(vector, top_k=8,
    practice_area='Personal Injury',
    jurisdiction='CA',
    doc_type='sol_rules')
  
```

Hoofdstuk 10. Structured database SQLite

Schema: SQLite vs tools

Tabel	Tool	Nut
clients	conflict_check	Belangen
attorneys	route_lead	Toewijzing
past_cases	estimate_value	Comps

Niet alles hoort in een vector DB. Entities met relaties en exacte lookups horen in SQLite.

Tabellen

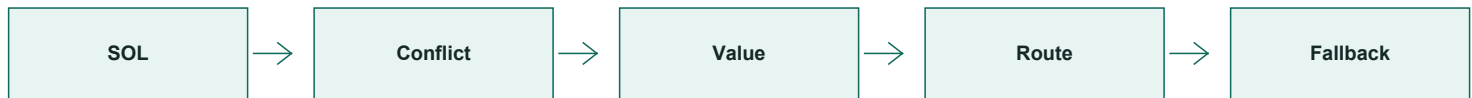
- clients — voor conflict_check (naam/opposing party)
- attorneys — voor route_lead (specialisatie, availability, jurisdictions)
- past_cases — voor estimate_case_value (comps)

Seed

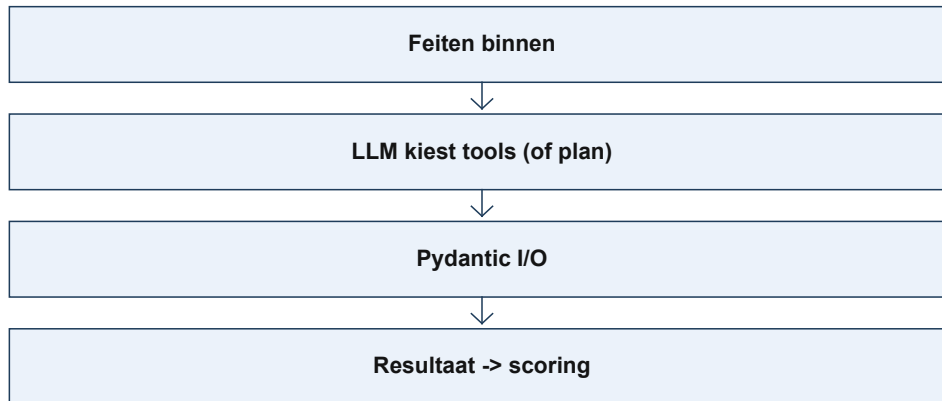
python db/init_structured_db.py maakt schema aan en vult vanuit kb/*.json. Conflict-demo's werken omdat seeded clients (zoals Elena Vasquez) bestaan.

Hoofdstuk 11. Agno tools (SOL, conflict, value, route)

Schema: Toolketen



Schema: Tool-aanroepstructuur



Tools zijn gewone Python-functies met Pydantic input/output, gewrapt als Agno `@tool` zodat de LLM ze kan aanroepen.

check_statute_of_limitations

Input: jurisdiction, case_type, incident_date. Output: valid, expires_in (dagen), explanation. Leest sol_tables.json.

conflict_check

Input: name, opposing_party. Zoekt in SQLite clients. Output: conflict bool + details.

estimate_case_value

Input: case_type, severity, damages. Blend van comps + stated damages. Output: estimate + range + explanation.

route_lead

Input: practice_area, priority. Kiest attorney op specialisatie/load. Output: attorney_name + rationale.

web_search_fallback (optioneel)

Lokale fallback wanneer KB tekort lijkt; in dit project geen live web.

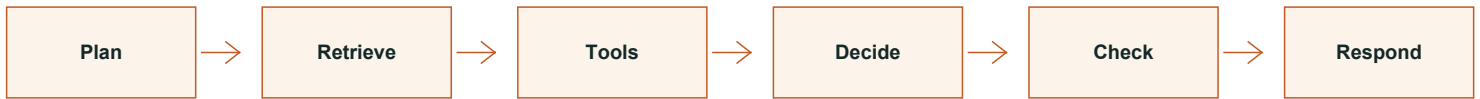
Agentic vs deterministisch

Met LLM: `Agent.run(..., tool_choice='auto')` laat het model tools kiezen. Zonder LLM (CI): de plan-fase bepaalt tools en roept endpoints direct aan. Ontbrekende toolresultaten worden deterministisch aangevuld voor betrouwbaarheid.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 12. De Intake Agent (plan → retrieve → tools → decide)

Schema: Agentische loop



Bij onvoldoende info: terug naar vragen / retrieve / escalate

Bestand: agents/intake_agent.py. Klasse IntakeAgent erft van Agno Agent.

Plan

Welke velden missen? Welke tools? Welke retrieval-query? Escaleren?

Retrieve

LanceDB semantic search + filters → citaties.

Tools

Agentic of deterministisch tool-aanroepen.

Decide

Lead score, viability, routing, next steps, confidence.

Self-check

Disclaimer? Citaten? Verboden taal? Onzekerheid?

Respond

Gebruikersbericht (LLM of template) + gestructureerde response.

Waarom deze loop?

De capstone eist expliciete agentic workflow. Door fasen te scheiden kun je monitoren (latency per stap), testen, en uitleggen wat er gebeurt — belangrijker dan één ondoorzichtige ‘black box’ prompt.

Hoofdstuk 13. Multi-turn interview met prospects

Schema: Interview-flow



Schema: Verplichte intake-velden

Veld	Voorbeeld
name	Alex Rivera
practice_area	Personal Injury
jurisdiction	CA
incident_date	2025-06-01
opposing_party	City Transit
damages	45000

De PDF vraagt om 'Interview prospective clients'. Daarom bestaat er een InterviewSession (agents/interview.py) en een UI-tab 'Interview (multi-turn)'.

Hoe werkt het?

- Agent start met welkom + disclaimer + eerste vraag
- Gebruiker antwoordt in chat
- Sessie vult IntakeFacts (heuristiek + optioneel LLM-extractie)
- Zolang verplichte velden missen: nieuwe vragen
- Als genoeg info (of 'screen now'): run_intake() screening
- Eindigt met samenvatting + guardrails + citaties

Verplichte velden

name, practice_area, jurisdiction, incident_date, opposing_party, damages

Quick analysis blijft bestaan

Voor demoscenari's en evaluatie is single-pass (plak tekst → analyse) handig. Interview is de conversatie-ervaring; quick analysis is de snelle pipeline-demo.

Hoofdstuk 14. Lead scoring & beslissingen

Schema: Scoring-pad



Schema: Beslissingsruimte

Decision	Betekenis
SCHEDULE_CONSULT	Plan intake
REVIEW	Menselijke check
REJECT	Niet aannemen

scoring/lead_scoring.py bevat `score_lead(context) → LeadScoreOutput`. Dit is deterministisch: zelfde context → zelfde beslissing.

Outputvelden

```

{
  "qualified": true,
  "lead_score": 78,
  "priority": "High",
  "decision": "SCHEDULE_CONSULT",
  "recommended_attorney": "Jordan Hale",
  "explanation": "..."
}
  
```

Beslissingen

- SCHEDULE_CONSULT — voldoende fit, geen harde blockers
- REVIEW — onzeker / incomplete / menselijke check nodig
- REJECT — harde blockers (bv. conflict of duidelijk verlopen SOL)

Harde regels (intuïtie)

- Conflict gevonden → meestal REJECT + escalatie
- SOL invalid → sterk negatief / reject pad
- Weinige acceptance matches → lagere score / review
- Hoge estimated value + goede fit → hogere priority

Hoofdstuk 15. Guardrails (juridische veiligheid)

Schema: Guardrail-stapel



Elke user-facing response moet aan deze eisen voldoen:

- Legal disclaimer aanwezig
- Nooit voorschrijvend juridisch advies (“you should sue” etc.)
- Escalate bij onzekerheid
- Citeer retrieved evidence (chunk_id, practice_area, doc_type)
- Weiger unsupported / verzonnen statutes of attorney profiles

Waar wordt dit afgedwongen?

- Agent instructions
- self_check fase
- respond() plakt disclaimer/escalatie/citaties indien nodig
- Evaluation check_guardrails()
- UI disclaimer componenten

Let op: Guardrails zijn een hard beoordelingspunt. Liever te vaak escaleren dan zelfverzekerd fout juridisch advies geven.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 16. Observability & Agno Monitoring

Schema: Observability-stack

Custom JSONL	latency/tools/scores
Metrics API	session/daily summary
Streamlit dash	Grafieken
Agno tracing	traces.db spans

De opdracht vraagt monitoring van sessies. LexIntake heeft twee lagen:

1) Custom metrics (monitoring/)

- logger.py — JSONL events, PII-sanitization
- metrics.py — tokens, cost, latency per stap, tool calls, retrieval hit rate, lead scores, escalaties, case values
- dashboard.py — Streamlit grafieken

2) Agno native tracing

- monitoring/agno_tracing.py roept setup_tracing() aan
- Traces landen in monitoring/traces.db
- Zichtbaar in dashboard sectie 'Agno Monitoring'
- Uit te zetten met LEXINTAKE_AGNO_TRACING=0 (CI doet dit)

```
python -m streamlit run monitoring/dashboard.py
```

Hoofdstuk 17. Evaluation harness

Schema: Evaluatieketen



Schema: Eval-dimensies (sample)

Dimensie	Meet
Retrieval	Hit rate
Grounding	Citations
Qualify	Label match
Guardrails	Compliance
Cost/Lat	Efficiency

evaluation/leads.csv bevat ~30 labeled synthetische leads. run_evaluation.py draait de agent en meet kwaliteit.

Acht dimensies

- Retrieval quality
- Grounding
- Qualification accuracy
- Case valuation
- Abstention behaviour
- Guardrails
- Cost & latency
- Provider comparison

Commands

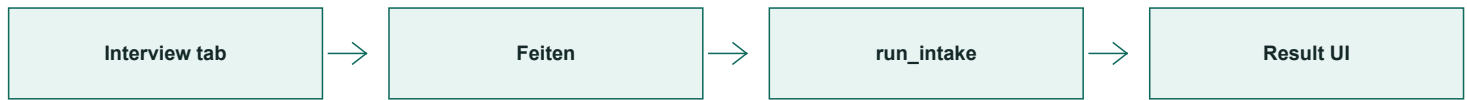
```
# Offline / CI-achtig
python evaluation/run_evaluation.py --providers local:deterministic --limit 5
```

```
# Live OpenAI (key vereist)
python evaluation/run_evaluation.py --providers openai:gpt-4.1 --limit 5
```

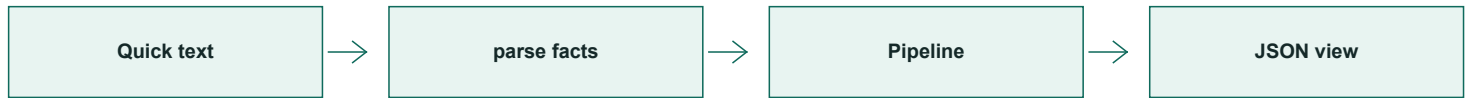
Rapport: docs/EVALUATION_REPORT.md. Perfecte qualificatie-accuracy is niet het doel; transparantie over grounding/guardrails wel.

Hoofdstuk 18. Streamlit UI & demo

Schema: UI Interview-pad



Schema: UI Quick-analysis-pad



App

```
python -m streamlit run ui/app.py
```

- Tab Interview — multi-turn gesprek
- Tab Quick analysis — plak case description → pipeline
- Sidebar met demoscenarió's
- Result viewer: score, decision, citaties, guardrails, tool JSON

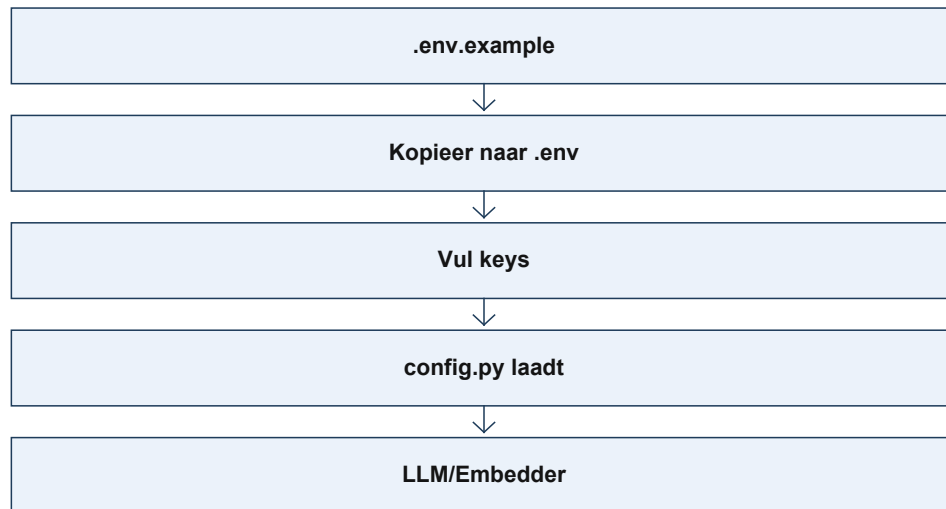
CLI demo (4 scenario's)

```
python ui/demo.py
```

Scenario's: valid PI → SCHEDULE_CONSULT; expired SOL → REJECT; conflict ACME → REJECT+escalate; uncertain immigration → REVIEW+escalate.

Hoofdstuk 19. Configuratie met .env

Schema: Config-flow



Secrets horen in .env (gitignored). Gebruik .env.example als template.

```
OPENAI_API_KEY=  
LEXINTAKE_EMBEDDING_PROVIDER=openai  
LEXINTAKE_EMBEDDING_MODEL=text-embedding-3-small  
LEXINTAKE_EMBEDDING_DIMS=1536  
LEXINTAKE_LLM_PROVIDER=openai  
LEXINTAKE_LLM_MODEL=gpt-4.1  
LEXINTAKE_AGNO_TRACING=1  
ANTHROPIC_API_KEY=  
GROQ_API_KEY=
```

Veiligheid

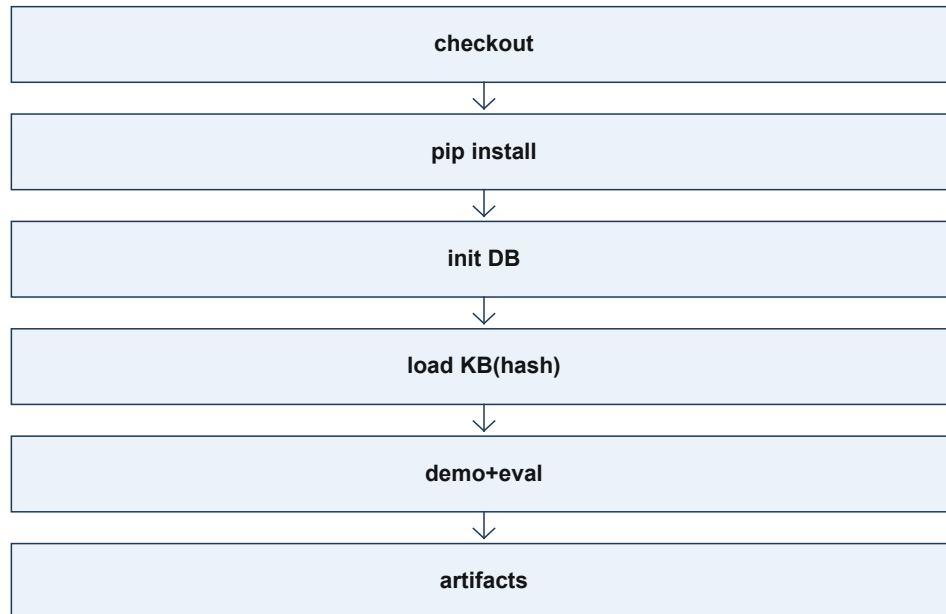
- Nooit keys in chat of commits plakken
- Gelekte key meteen intrekken en roteren
- config.py laadt dotenv; libraries lezen OPENAI_API_KEY uit env

Hoofdstuk 20. Git-workflow, PR's en CI

Schema: Git + CI flow



Schema: CI smoke-test stappen



Branches

- main — protected production
- develop — integratie
- feature/* — features

CI (.github/workflows/ci.yml)

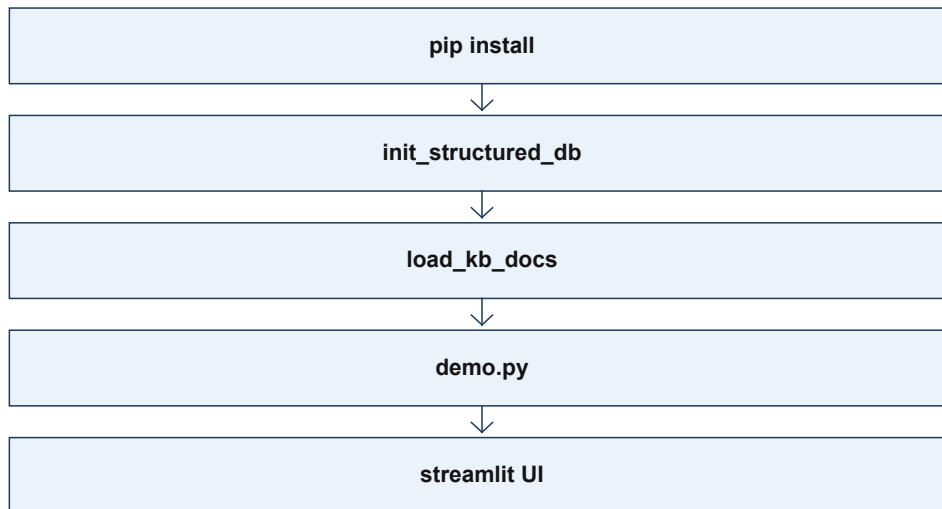
Draait op pull_request naar main, job smoke-test, volledig offline:

- Python 3.11 + pip install
- init_structured_db
- load_kb_docs (hash embeddings)
- ui/demo.py
- evaluation --limit 5 (local)
- upload logs als artifacts

main vereist status check smoke-test. Daardoor blokkeert een falende CI een merge.

Hoofdstuk 21. Stap-voor-stap: systeem starten

Schema: Startvolgorde



Eerste keer

```
cd C:\Users\steve\LexIntake
pip install -r requirements.txt
copy .env.example .env
# Zet OPENAI_API_KEY in .env
python db/init_structured_db.py
python db/load_kb_docs.py
python ui/demo.py
python -m streamlit run ui/app.py
```

Alleen offline

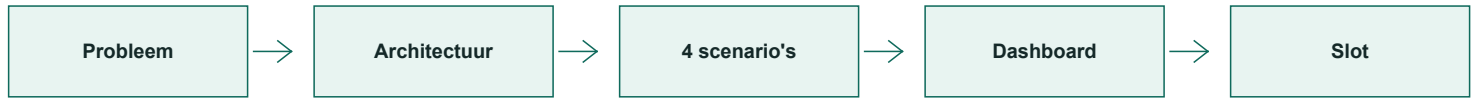
```
$env:LEXINTAKE_EMBEDDING_PROVIDER='hash'
$env:LEXINTAKE_EMBEDDING_DIMS='256'
$env:LEXINTAKE_LLM_PROVIDER='local'
$env:LEXINTAKE_AGNO_TRACING='0'
python db/load_kb_docs.py
python ui/demo.py
```

Handige checks

- Demo 4/4 PASS
- Interview tab stelt vragen en eindigt met screening
- Dashboard laadt metrics / Agno traces
- Geen .env in git status

Hoofdstuk 22. Demo-video voorbereiden

Schema: Video-storyboard



Gebruik docs/DEMO.md. Suggestie voor 3–5 minuten:

- 0:00 probleem + “geen chatbot, wel agentic RAG”
- 0:20 repo/architectuur in 30 seconden
- 0:50 live UI: valid PI scenario
- 1:40 SOL expired → REJECT
- 2:10 conflict → escalate
- 2:40 uncertain → REVIEW
- 3:10 monitoring dashboard
- 3:40 evaluation highlight (grounding/guardrails)
- 4:10 afsluiting + disclaimer

Toon altijd de disclaimer on-screen. Upload naar Loom/YouTube unlisted en zet de link in de README.

Hoofdstuk 23. Troubleshooting & veelgemaakte fouten

Schema: Debug-matrix

Symptoom	Check
Import error	venv + requirements
Key missing	.env / local mode
Dim mismatch	herlaad KB
CI rood	hash+local offline

ModuleNotFoundError llm / embeddings

Zorg dat je vanuit repo-root draait en requirements geïnstalleerd zijn.

OPENAI_API_KEY missing

Zet key in .env of schakel naar hash/local providers.

Dimensie mismatch LanceDB

Normaal bij switch hash↔OpenAI. load_kb_docs herbout de tabel.

Conflict-demo faalt

init_structured_db opnieuw draaien zodat Elena Vasquez geseeded is.

CI rood

Lokaal nabootsen met hash/local env vars en demo + eval --limit 5.

IntakeFacts validation error

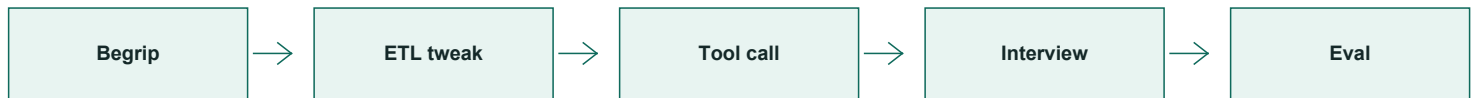
Dual imports; recente fix herbindt naar één model via model_dump.

Key gelekt in chat

Intrekken, nieuwe maken, nooit opnieuw plakken in issues/PR's.

Hoofdstuk 24. Oefeningen om te oefenen

Schema: Oefenparcours



Oefening A — Begrip

Leg in 10 zinnen uit aan een klasgenoot wat LexIntake doet zonder code te tonen. Gebruik de woorden: lead, RAG, tool, guardrail, escalatie.

Oefening B — ETL

Voeg één FAQ-regel toe in kb/faqs.md, run `load_kb_docs`, en toon dat retrieval de nieuwe chunk kan vinden.

Oefening C — Tools

Roep `check_statute_of_limitations` aan voor CA/Personal Injury met een oude en een recente `incident_date`. Vergelijk `valid/expires_in`.

Oefening D — Interview

Doe een interview in de UI waarbij je expres jurisdiction weglaat. Controleer dat de agent doorvraagt.

Oefening E — Eval

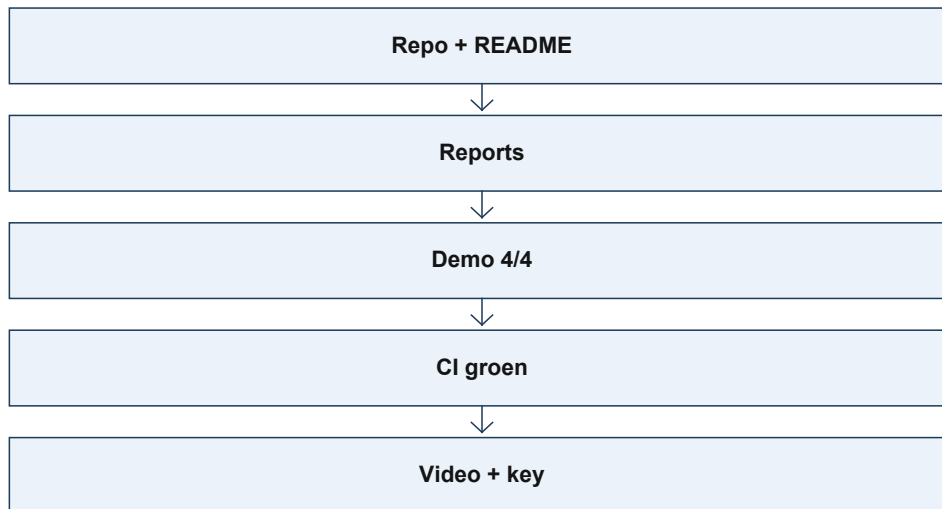
Draai `evaluation --limit 5 offline` en noteer `grounding + guardrail compliance`.

Oefening F — Uitlegscore

Neem één lead en schrijf met de hand waarom de decision `SCHEDULE/REVIEW/REJECT` is, op basis van `SOL/conflict/value/acceptance`.

Hoofdstuk 25. Glossarium & checklist oplevering

Schema: Oplevervolgorde



Mini-glossarium

- Agno — agent framework
- LanceDB — lokale vector DB
- chunk_id — stabiele id van een tekstbrok
- upsert — insert-or-update
- abstention — systeem onthoudt zich / escaleert
- grounding — antwoorden steunen op retrieved evidence
- smoke-test — snelle CI-check dat het systeem niet kapot is

Opleverchecklist

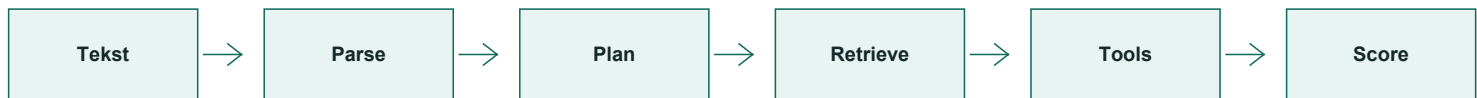
- Repo publiek/deelbaar met instructor
- README klopt (setup + disclaimer)
- Design Report + Evaluation Report aanwezig
- Dashboard draait
- Demo 4/4 groen
- CI groen op main PRs
- main protected
- OPENAI key lokaal gezet (niet gecommitt)
- Demo-video opgenomen + link toegevoegd
- Je kunt de architectuur hardop uitleggen

Laatste advies

Beoordelaars kijken naar: werkt het end-to-end, is het agentic/RAG echt, zijn guardrails zichtbaar, is Git professioneel, en kun jij het uitleggen. Perfecte ML-metrics zijn minder belangrijk dan een helder, veilig intake-verhaal.

Hoofdstuk 26. Werkvoorbeeld end-to-end: Valid Personal Injury

Schema: Runtime end-to-end



Dit hoofdstuk volgt één lead van ruwe tekst tot beslissing. Lees het alsof je meekijkt in de runtime.

Input

Rear-end collision in CA, clear liability, \$45k damages, incident 6 months ago.

Stap 1 — Fact parsing

parse_case_description herkent 'rear-end/collision' als Personal Injury, staat CA, damages 45000, incident_date ≈ vandaag minus 6*30 dagen, severity medium/high afhankelijk van keywords, naam Demo Prospect.

Stap 2 — Plan

Omdat jurisdiction, case_type en incident_date bekend zijn, plant de agent check_statute_of_limitations. Met name+party komt conflict_check. Met damages komt estimate_case_value. Met practice_area komt route_lead. Retrieval staat aan.

Stap 3 — Retrieve

De query mix practice area, CA, narrative en woorden als acceptance/settlement. LanceDB geeft chunks terug (acceptance_criteria, sol_rules, past_case, faq). Die chunks worden citaties.

Stap 4 — Tools

- SOL CA/PI: meestal valid=True, expires_in positief (2-jaar regel)
- Conflict: geen match → conflict=False
- Value: blend comps + 45k → schatting in de orde van tonnen kan voorkomen door sparse comps; interpretatie: richtinggevend, niet exact
- Route: attorney met PI-specialisatie, bv. Jordan Hale

Stap 5 — Scoring

score_lead ziet geen conflict, SOL ok, practice match → qualified True, priority High, decision SCHEDULE_CONSULT.

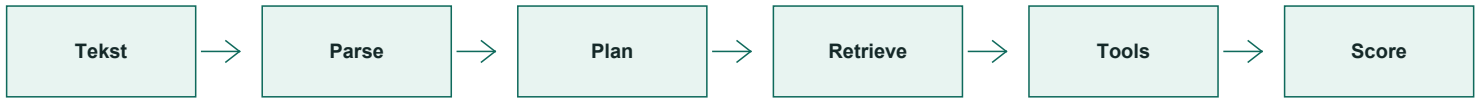
Stap 6 — Guardrails

Response bevat disclaimer, citaties, geen 'you should sue'. Escalatie niet nodig.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 27. Werkvoorbeeld: verlopen SOL

Schema: Runtime end-to-end



Slip-and-fall in NV, incident 4 years ago, moderate injuries.

Parser: Personal Injury, NV, incident_date ver in het verleden. SOL-tool: valid=False, expires_in negatief. Scoring duwt naar REJECT. Uitleg noemt statute/expiration. Dit toont dat tools niet cosmetisch zijn: ze veranderen de beslissing.

In de demo checkt verify_scenario dat de explanation SOL-taal bevat. Zo bewijs je grounding + tool-effect in één scenario.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 28. Werkvoorbeeld: conflict of interest

Schema: Runtime end-to-end



Employment discrimination claim in CA, opposing party is ACME Corp.

De parser map't dit bewust naar client Elena Vasquez (seeded conflict case) wanneer employment+ACME voorkomt. conflict_check vindt een match in SQLite. Beslissing: REJECT of minstens escalatie. Belangrijk lespunt: structured DB is essentieel; vector search alleen is niet genoeg voor exacte identity checks.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 29. Werkvoorbeeld: onzekere immigratiezaak

Schema: Runtime end-to-end

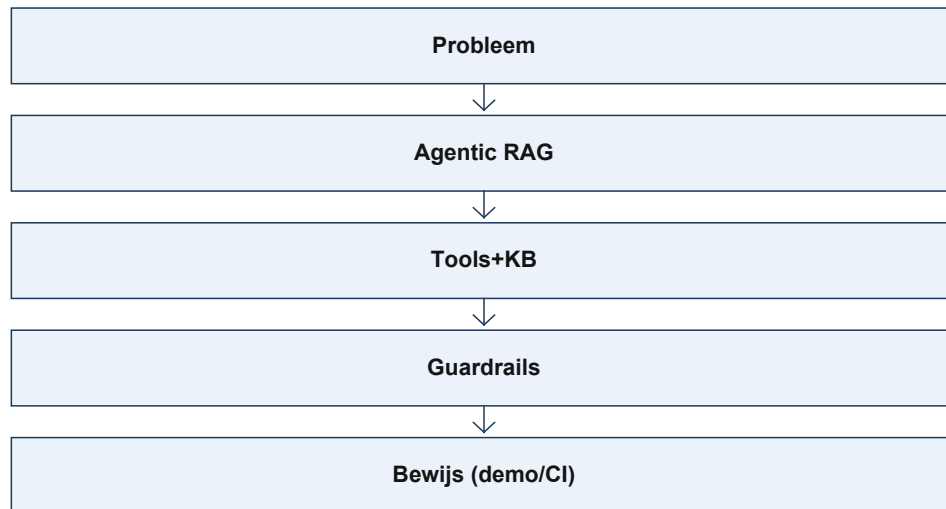


Immigration matter with unclear facts, missing dates, missing jurisdiction.

Veel velden ontbreken. Runner markeert uncertain. Zelfs als scoring niet hard reject, forceert UI/runner REVIEW + escalatie. Dit is abstention behaviour: liever menselijke review dan overconfident automatisering.

Hoofdstuk 30. Hoe leg je LexIntake uit in 3 minuten?

Schema: Pitch-structuur



Gebruik dit spreekscript:

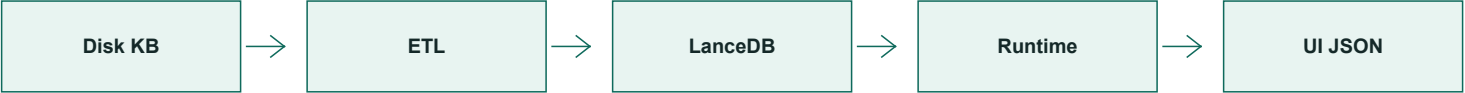
- Probleem: kantoren verdrinken in leads; paralegals filteren handmatig.
- Oplossing: agentic RAG screener — niet chatbot.
- Kennis: synthetische KB + ETL naar LanceDB + SQLite entities.
- Agent: plant, retrieve't, roept tools, scoort, self-checkt.
- Veiligheid: disclaimer, citaties, escalatie, geen legal advice.
- Bewijs: demo 4 scenario's, evaluation metrics, CI smoke, GitHub flow.

Sluit af met: 'The system recommends intake actions for staff; it does not advise clients. This is not legal advice.'

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 31. Dataflow in detail (van disk tot antwoord)

Schema: Dataflow 30.000 ft



1) Bestanden in kb/ liggen op disk. 2) `extract_all` leest ze. 3) `clean` normaliseert. 4) `dedupe` berekent `content_hash`. 5) `chunk` snijdt tekst en maakt `chunk_id`. 6) `metadata` hangt `practice_area/doc_type/jurisdictions`. 7) `embedder` maakt vectoren. 8) `upsert_kb_docs` schrijft LanceDB. 9) Bij runtime embedt de agent de query. 10) `search_kb_docs` rangschikt. 11) Tools lezen JSON/SQLite. 12) `score_lead` beslist. 13) `respond` bouwt tekst + JSON voor UI.

Als je dit rijtje kunt opschrijven zonder te spieken, begrijp je het project.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Hoofdstuk 32. Provider-agnostic ontwerp

Schema: Provider-scheiding

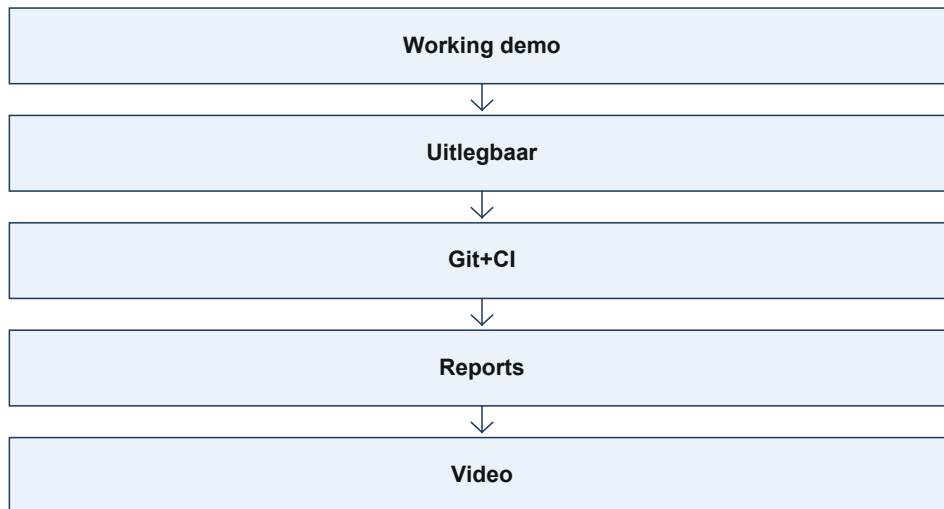
Laag	Keuze
LLM	openai/anthropic/groq/
Embed	openai/hash
Logic	onveranderd

config.py + agents/llm.py + etl/embeddings.py scheiden ‘welke vendor’ van ‘welke business logic’. Daardoor kun je OpenAI, Anthropic of Groq kiezen voor LLM, en openai/hash voor embeddings, zonder tools/scoring te herschrijven.

CI forceert local/hash zodat grading/instructor-runs niet van jouw creditcard afhangen. Live demos gebruiken OpenAI voor betere retrieval-narratieven.

Hoofdstuk 33. Wat is ‘goed genoeg’ voor grading?

Schema: Grading-prioriteiten



- Working demo die 4 scenario's correct toont
- Je kunt architectuur en guardrails uitleggen
- Git history + PR + CI zichtbaar
- Reports aanwezig en eerlijk over zwakke metrics (valuation)
- Video die het verhaal toont

Zwakke case-value accuracy is oké als je uitlegt waarom (sparse comps) en hoe je het zou verbeteren. Hallucinaties zonder citaties zijn niet oké.

Hoofdstuk 34. Veiligheids- en ethiekles

Schema: Veiligheidslagen

No advice	Geen directives aan prospect
Escalate	Twijfel -> mens
Cite	KB evidence
Secrets	Keys niet in git

Legal AI heeft asymmetrisch risico: een fout advies kan schade veroorzaken. Daarom: geen directives aan prospects, wel interne screening-samenvattingen; escalatie bij twijfel; logging zonder ruwe PII waar mogelijk; keys niet in git.

Als iemand vraagt 'Moet ik aanklagen?' is het juiste systeemgedrag: weigeren juridisch advies te geven en doorverwijzen naar een licensed attorney.

Hoofdstuk 35. Uitbreidingen als je tijd over hebt

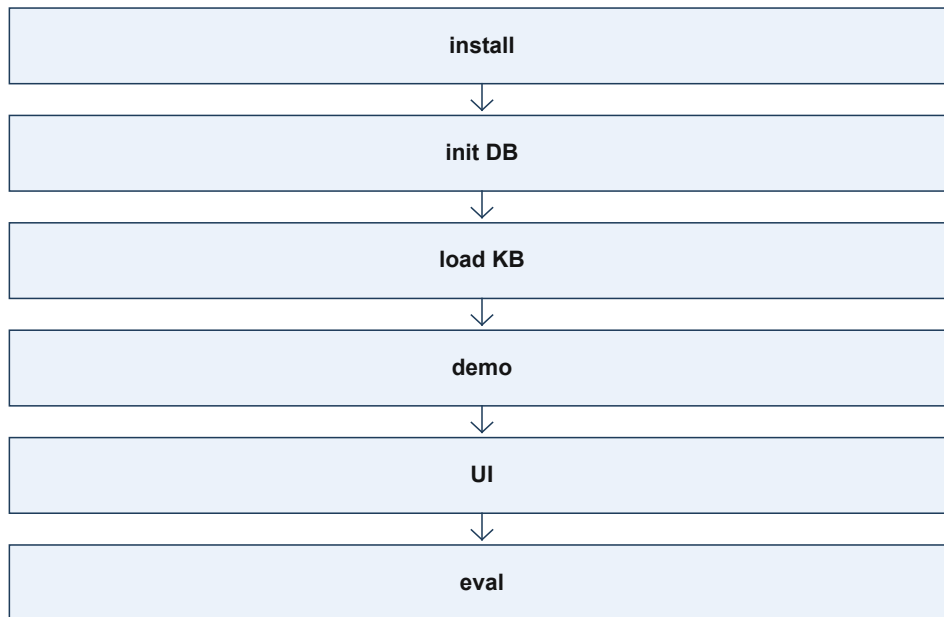
Schema: Stretch roadmap



- Corrective RAG: als citaties zwak zijn, herbreek query
- Multi-agent: aparte agents voor SOL vs conflict vs narration
- HITL console: paralegal keurt SCHEDULE_CONSULT goed/af
- Persistent memory: herhaalde callers herkennen
- Rijkere SOL-tabellen met numerieke years i.p.v. alleen free text
- Meer past_cases per practice area voor betere valuation

Hoofdstuk 36. Cheatsheet: commands die je moet kennen

Schema: Command-volgorde



```
pip install -r requirements.txt
copy .env.example .env
python db/init_structured_db.py
python db/load_kb_docs.py
python ui/demo.py
python -m streamlit run ui/app.py
python -m streamlit run monitoring/dashboard.py
python evaluation/run_evaluation.py --providers local:deterministic --limit 5
python docs/generate_leerboek_pdf.py
```

Hoofdstuk 37. Cheatsheet: bestanden die je moet kunnen aanwijzen

Schema: Bestanden om aan te wijzen

```
agents/intake_agent.py <- loop
tools/*.py             <- acties
scoring/lead_scoring.py
db/load_kb_docs.py
ui/app.py
.github/workflows/ci.yml
```

- Opdrachtbegrip -> docs/DESIGN_REPORT.md + dit leerboek
- KB -> kb/*.json + faqs.md
- ETL -> etl/*.py
- Vector load -> db/load_kb_docs.py + db/lancedb_store.py
- SQLite -> db/sqlite_db.py + db/sql/schema.sql
- Tools -> tools/*.py
- Agent -> agents/intake_agent.py
- Interview -> agents/interview.py
- Scoring -> scoring/lead_scoring.py
- UI -> ui/app.py
- Eval -> evaluation/run_evaluation.py
- CI -> .github/workflows/ci.yml
- Config -> config.py + .env

Hoofdstuk 38. Zelftest (antwoord ondersteboven in je hoofd)

Schema: Zelftest-kaart

Vraag	Focus
RAG vs LLM?	Grounding
SQLite waarom?	Exact match
5 fasen?	Loop
CI offline?	hash/local

- Wat is het verschil tussen RAG en 'gewoon een LLM vragen'?
- Waarom bestaat SQLite naast LanceDB?
- Noem de 5 agent-fasen in volgorde.
- Welke beslissingen kan score_lead geven?
- Hoe dwingt CI af dat er geen API-keys nodig zijn?
- Wat moet er altijd in een user-facing antwoord staan?
- Welk scenario toont conflict detection?
- Waar landen Agno traces?

Als je deze acht vragen soepel beantwoordt, ben je klaar om te presenteren.

UI	Agent	RAG	Tools	Score	Guard
----	-------	-----	-------	-------	-------

Einde van het LexIntake Leerboek

Je hebt nu een route van probleemstelling naar implementatie, runtime, evaluatie en oplevering. Gebruik de repo als lab: wijzig iets kleins, draai demo/eval, en observeer het effect.

This is not legal advice. Consult a licensed attorney for legal guidance.
Dit document is uitsluitend bedoeld om de capstone-opdracht te leren.